Sentence-level embeddings

Prof. Luca Cagliero
Dipartimento di Automatica e Informatica
Politecnico di Torino

Politecnico
di Torino

Lecture goals

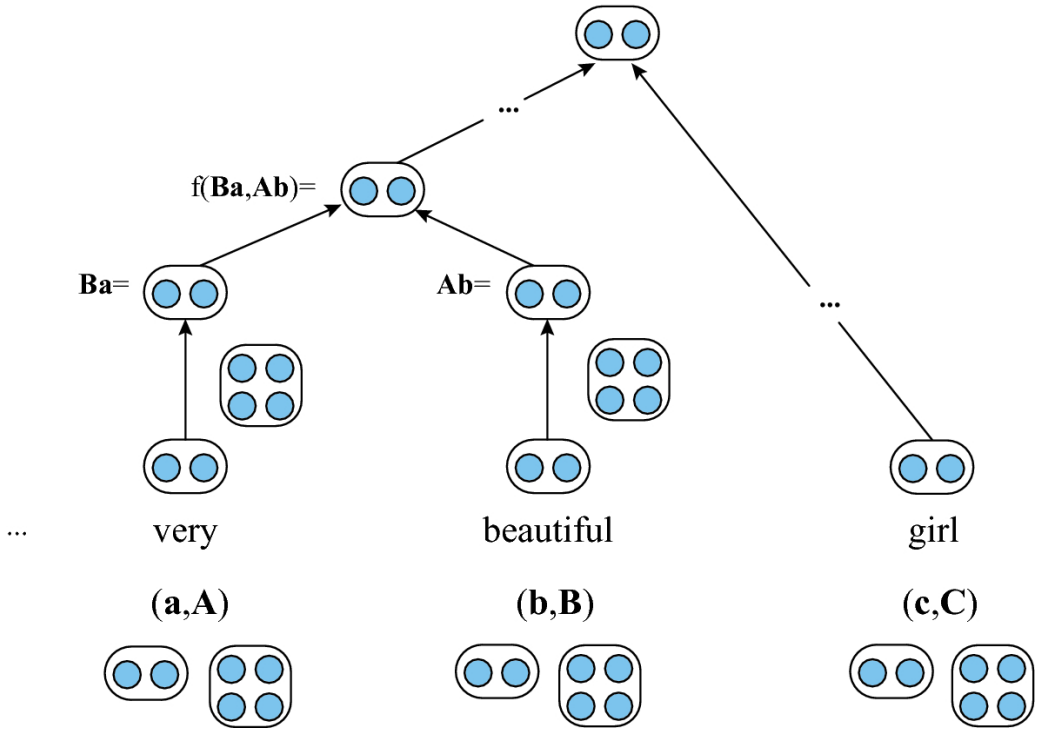
- Compositionality
  - From word to sentences
- Sentence-level embedding models
  - Doc2Vec
  - Sent2Vec
  - Skip-thought vectors
  - InferSent
  - Universal Sentence Encoder
- Embedding model evaluation
  - Intrinsic evaluation
  - Extrinsic evaluation
- Bias in word embedding models

Lecture goals

- **Compositionality**
  - **From word to sentences**
- Sentence-level embedding models
  - Doc2Vec
  - Sent2Vec
  - Skip-thought vectors
  - InferSent
  - Universal Sentence Encoder
- Embedding model evaluation
  - Intrinsic evaluation
  - Extrinsic evaluation
- Bias in word embedding models

Compositionality

- How to combine small textual units into higher-level ones
  - E.g., words into sentences, sentences into paragraphs
- The meaning of a complex expression is determined by the meanings of its constituent expressions and the rules used to combine them
- Crucial for NLP



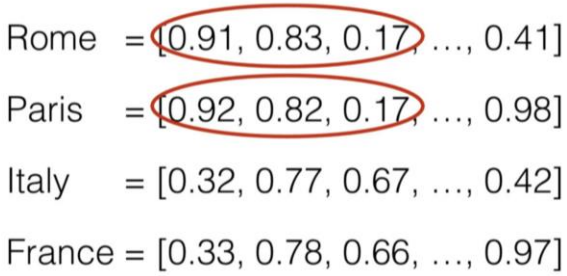
Embeddings in Natural Language Processing Theory and Advances in Vector Representation of Meaning. Mohammad Taher Pilehvar, Jose Camacho-Collados. Morgan & Claypool Publisher.

Bag-of-word

- Traditional method to represent long pieces of text through word-level features
- Suitable for word-level occurrence-based models
  - Word-document representation
- Unsuitable for sentence- and document-level compositions
  - Insensitive to word ordering
  - Disregards word repetitions

From words to sentences

- **Word embeddings:** dense vectors that captures the semantics of words. They enable word-to-word similarity computation.

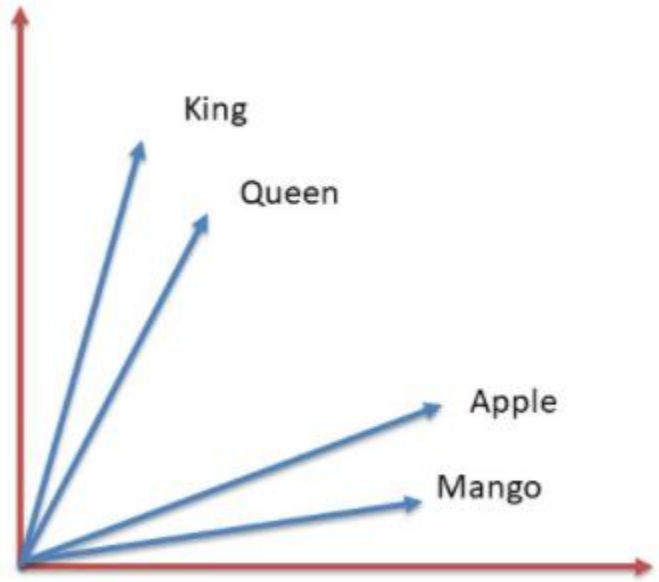


https://speakerdeck.com/marcobonzanini/word-embeddings-for-natural-language-processing-in-python-at-london-python-meetup?slide=22

- Applications in several NLP tasks (e.g., topic analysis)

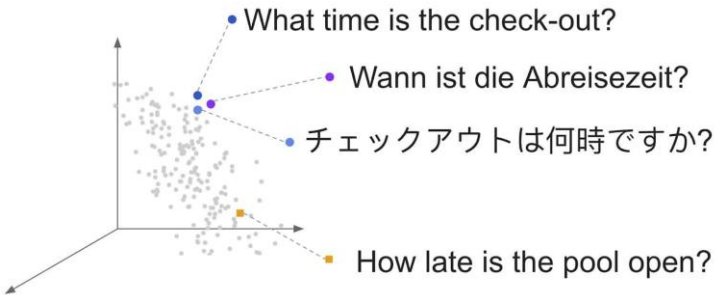
Word-level embeddings

- Outperform occurrence-based text representations in many NLP tasks
- Unsuitable for sentence- and document-level composition



From words to sentences

- **Sentence embeddings:** dense vectors the semantic meaning of the entire sentence.

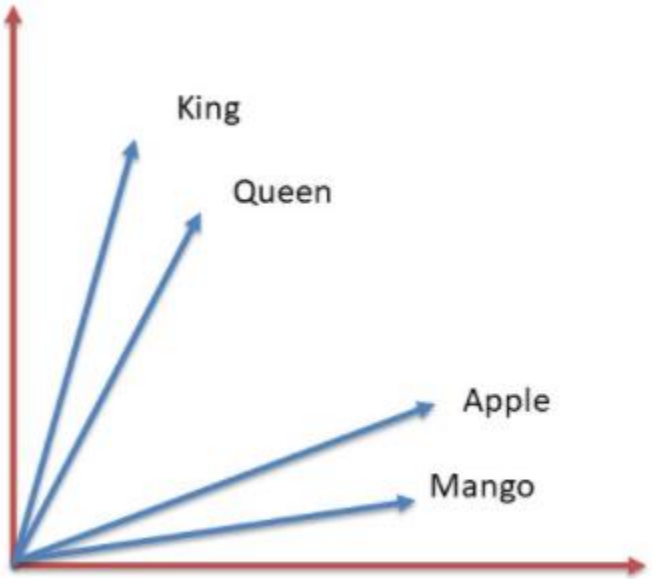


Source: https://megagon.ai/blog/emu-enhancing-multilingual-sentence-embeddings-with-semantic-similarity/

- There are several methods to aggregate words into sentences
  - Both **static** and **dynamic** embeddings
    - For now we focus on static embeddings

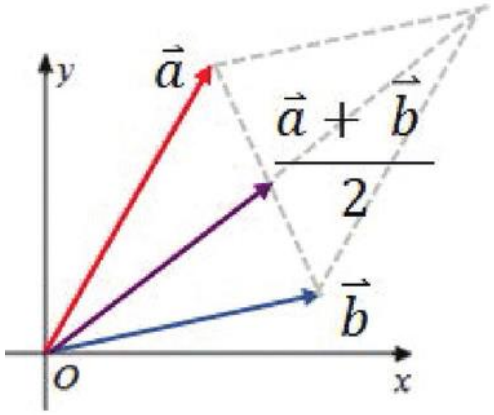
Sentence-level embeddings

- Direct training of vector representations of text on sentences
- Understanding of isolated sentences without having to rely solely on their components

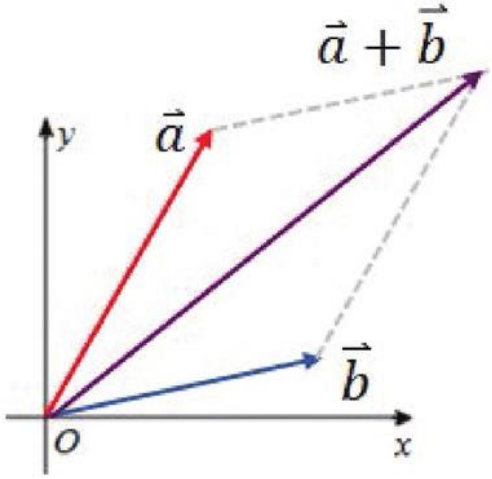


From word to sentence embeddings

- Two-step process
  1. Infer word vectors
  2. Derive sentence vectors from word vectors



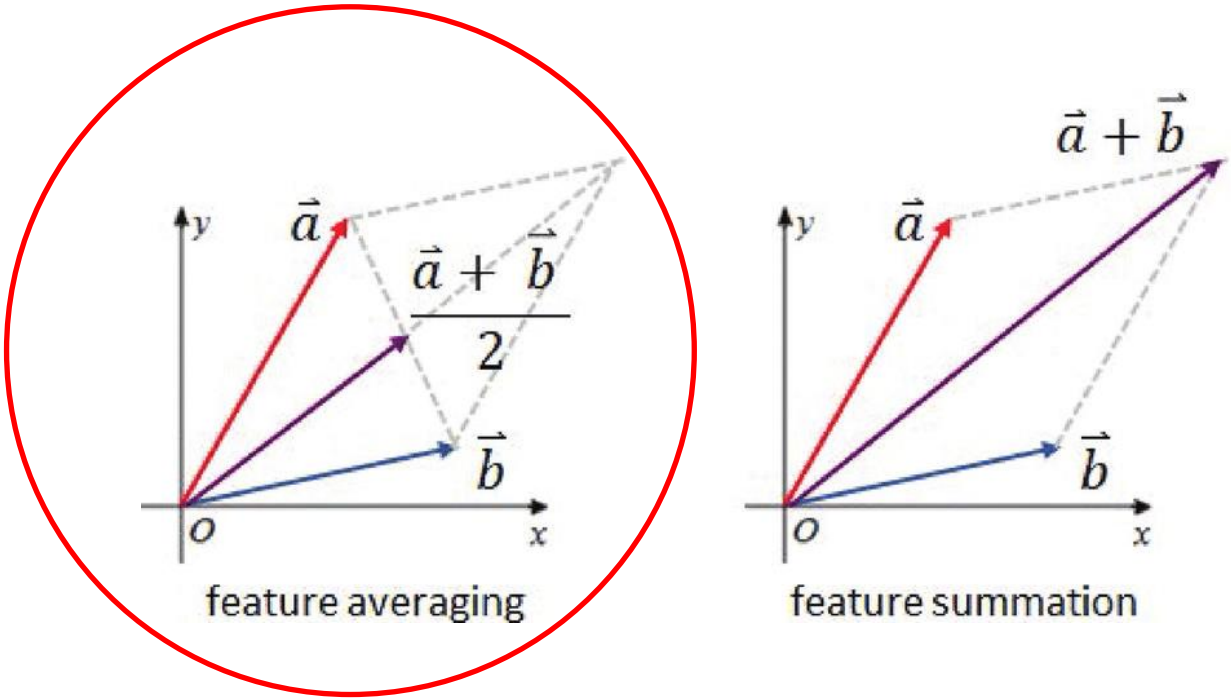
feature averaging



feature summation

From word to sentence embeddings

- Perform vector arithmetic on all the vectors corresponding to the words of the document
  - E.g., feature averaging
- Summarize them into a single vector in the same embedding space



Drawbacks of feature averaging (solved by sentence embeddings)

- In the fixed-size context the network ignores sentence-level word relations
- Word vector combinations are independent of the network target

Drawbacks of feature averaging (not solved by non-contextualized models yet)

- Word ordering matters
- Example
  - I'm going to study Math instead of English
  - I'm going to study English instead of Math
- The network has to encode positional information

Lecture goals

- From word to sentences
- **Sentence-level embedding models**
  - Doc2Vec
  - Sent2Vec
  - Skip-thought vectors
  - InferSent
  - Universal Sentence Encoder
- Embedding model evaluation
  - Intrinsic evaluation
  - Extrinsic evaluation
- Bias in word embedding models

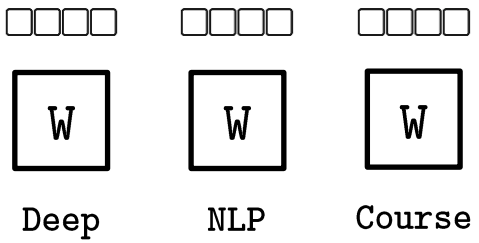
The sentence embedding approach

- Encode sentence-level information directly using network training
- Allow the network to learn how to combine word vectors into document embeddings
- Two complementary approach
  - **Unsupervised sentence encoding**: learn generic, distributed sentence encoding from plain text
    - E.g., Doc2Vec, Sent2Vec, Skip-Thought
  - **Supervised sentence encoding**: learn sentence encodings from labeled data
    - Transfer learning paradigm

Doc2Vec

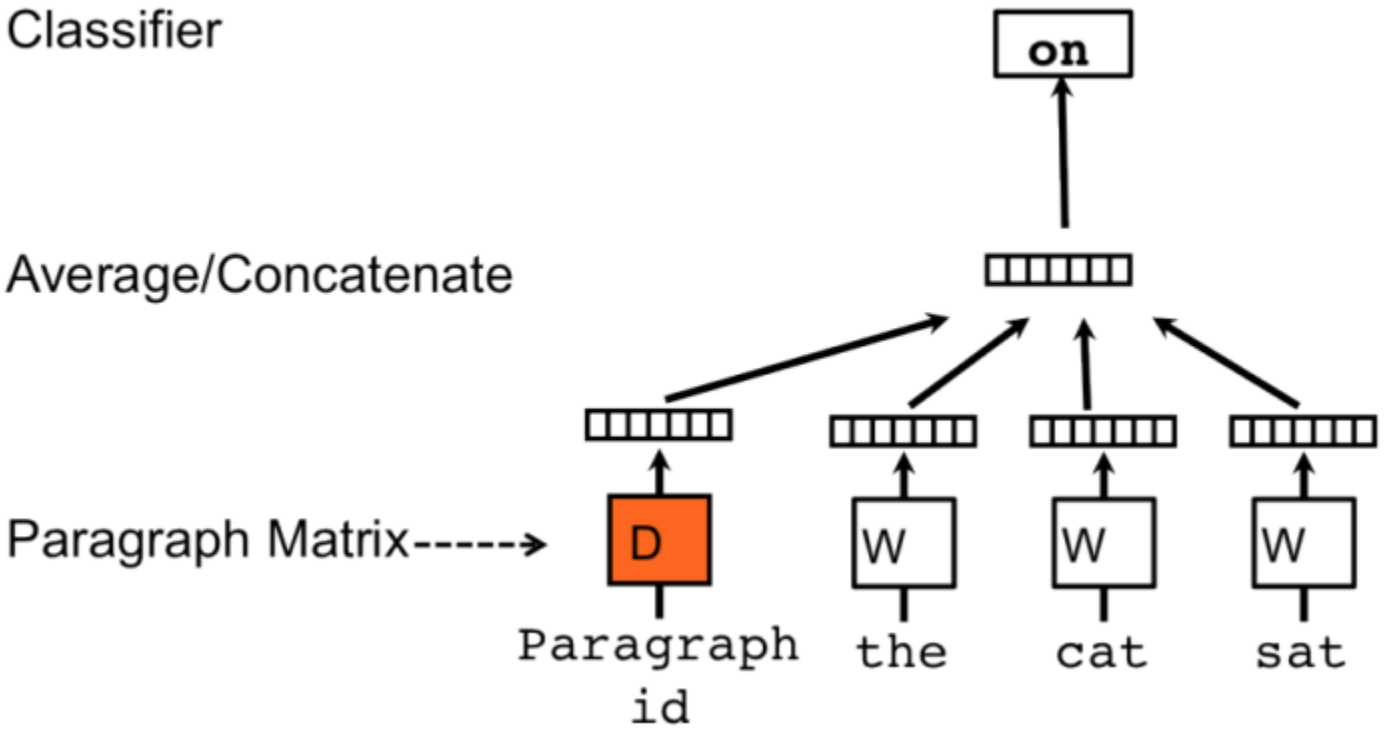
- Also known as *Paragraph2Vec*
- First attempt to generalize Word2Vec to longer sequences of words
  - Relies on the assumption according to which word's meaning is given by the words that appear nearby
- Based on a paragraph vector model
- Two main algorithm's variants:
  - **Distributed Memory model (PV-DM)**
  - **Distributed Bag of Words (PV-DBOW)**

What is possible to obtain with standard Word2Vec model



Quoc V. Le, Tomás Mikolov. Distributed Representations of Sentences and Documents. ICML 2014: 1188-1196

Distributed Model of Paragraph Vector (PV-DM)



https://www.topbots.com/ (latest access: June 2021)

Distributed Model of Paragraph Vector (PV-DM)

- Every paragraph is mapped to a unique vector
  - represented by a column in a matrix D
- Every word is also mapped to a unique vector
  - represented by a column in matrix W
- The paragraph token can be thought of as another word
  - It acts as a memory that remembers either what is missing from the current context or the topic of the paragraph

Quoc V. Le, Tomás Mikolov. Distributed Representations of Sentences and Documents. ICML 2014: 1188-1196

Distributed Model of Paragraph Vector (PV-DM)

- Prediction task analogous to CBOW
  - Predict the next word given the context
    - The context words are the preceding words only!
    - The surrounding context is encoded by the paragraph id
- The paragraph vector and word vectors are averaged or concatenated to predict the next word in a context

Quoc V. Le, Tomás Mikolov. Distributed Representations of Sentences and Documents. ICML 2014: 1188-1196

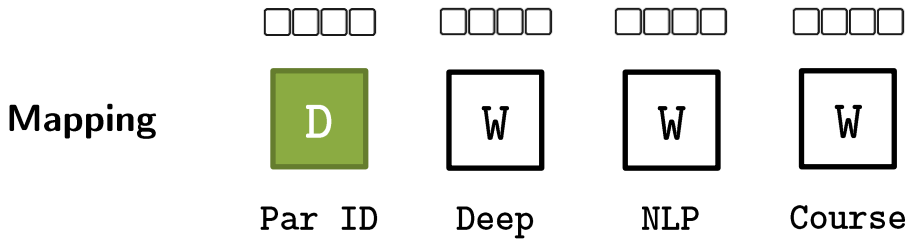
Distributed Model of Paragraph Vector (PV-DM)

- The contexts are fixed-length and sampled from a sliding window over the paragraph
- The paragraph vector is shared across all the contexts generated from the same paragraph but not across paragraphs

Quoc V. Le, Tomás Mikolov. Distributed Representations of Sentences and Documents. ICML 2014: 1188-1196

PV-DM

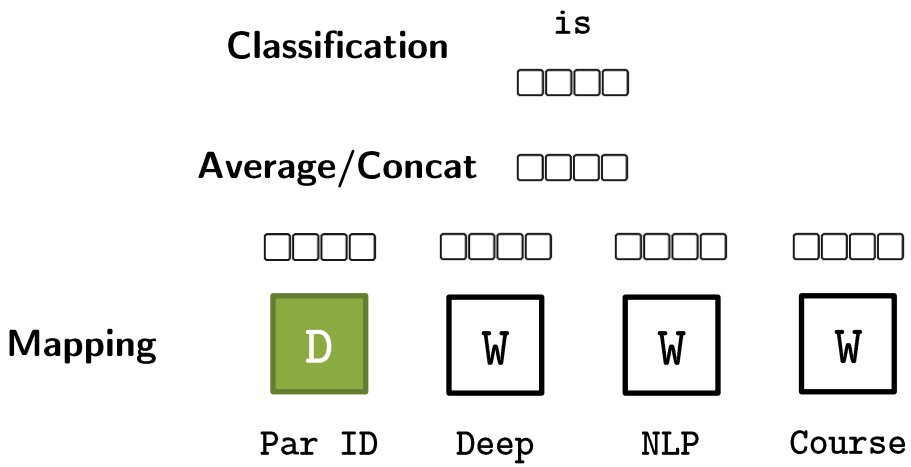
- **Distributed Memory model (similar to CBOW)**
  - Every paragraph is mapped to a unique vector (matrix **D**)
  - Every word is mapped to a unique vector (matrix **W**)



Paragraph: Deep NLP course is live

PV-DM

- **Distributed Memory model (similar to CBOW)**
  1. Every paragraph is mapped to a unique vector (matrix **D**)
  2. Every word is mapped to a unique vector (matrix **W**)
  3. The **paragraph vector** and **word vectors** are averaged or concatenated to predict the next word in a context.

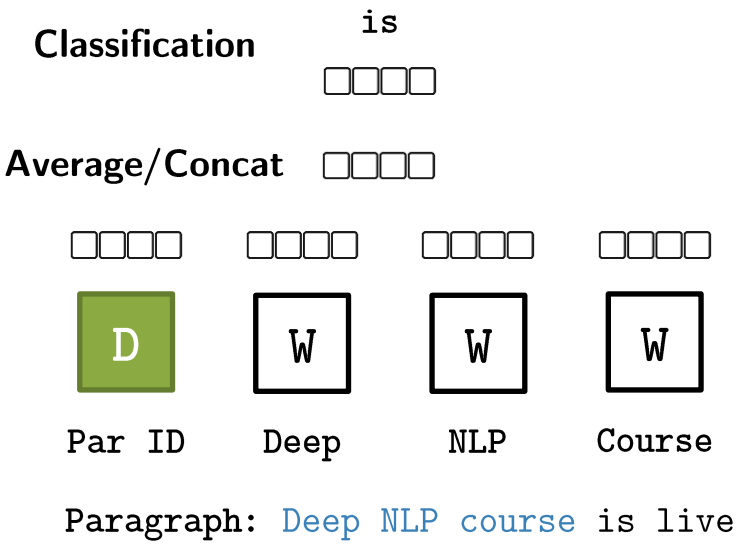


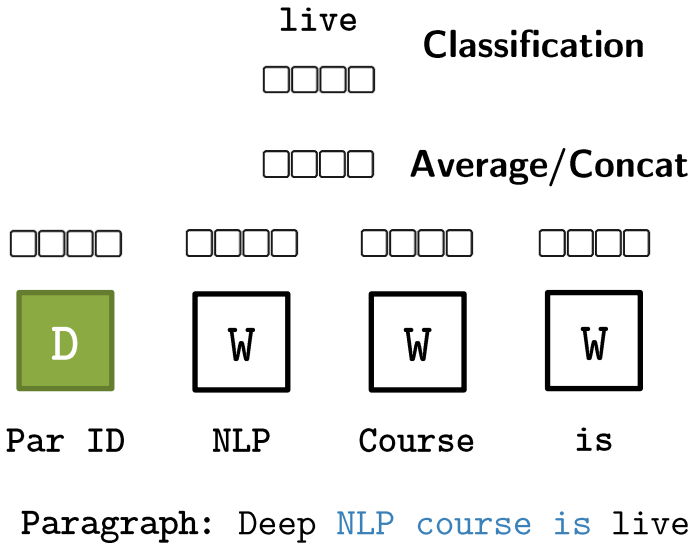
Paragraph: Deep NLP course is live

PV-DM

Paragraph Vector: can be thought as an additional word. It acts as a memory that remembers what is missing from the current context ("topic" of the paragraph)

Context: sliding window over the paragraph. The paragraph vector is shared across all contexts generated from the same paragraph (not across paragraphs)





PV-DM

- **Classification objective:** paragraph vectors and word vectors are trained using stochastic gradient descent and the gradient is obtained via backpropagation
- During training, at each step of stochastic gradient descent:

1. **Sample** a fixed-length context from a random paragraph
2. **Compute the error** gradient from the network
3. **Use the gradient to update the parameters** in the embedding model.

PV-DM

- **Classification objective:** paragraph vectors and word vectors are trained using stochastic gradient descent and the gradient is obtained via backpropagation.
- At prediction time the embedding for a new paragraph is obtained by fixing the parameters for word vectors (**W**) and classification
- After training, paragraph vectors can be used as features for the paragraph. It is possible to **feed these features** directly **to machine learning techniques** such as logistic regression, support vector machines or K-means.

Distributed Model of Paragraph Vector (PV-DM)

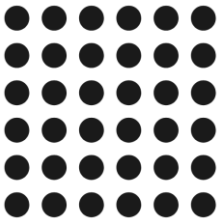
- When you predict the vector of a new paragraph a new vector is added to the matrix D (new column)
- All the weights for words and classification are fixed
- D can be used as features for subsequent tasks

Quoc V. Le, Tomás Mikolov. Distributed Representations of Sentences and Documents. ICML 2014: 1188-1196

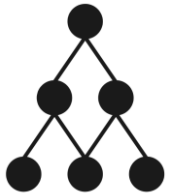
PV-DM

Tr: training phase - Te: test/inference phase

Word matrix (**W**):
- Learned in Tr
- Fixed in Te

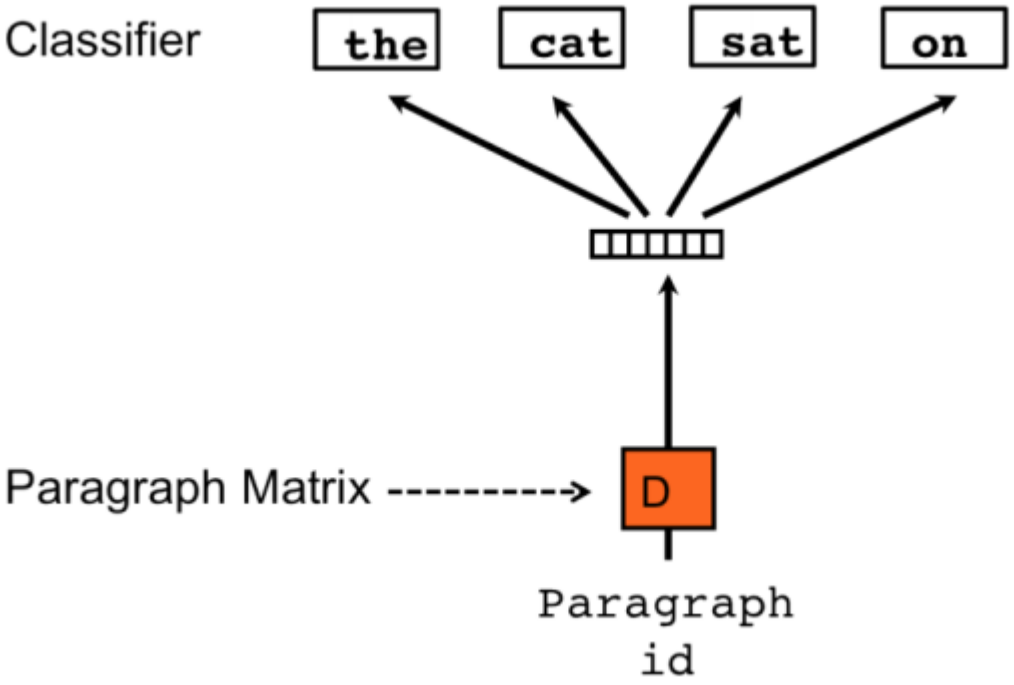


Paragraph matrix (**D**):
- Learned in Tr
- **Learned** in Te



Classification weights:
- Learned in Tr
- Fixed in Te

Distributed Bag-of-Words (PV-DBOW)



https://www.topbots.com/ (latest access: June 2021)

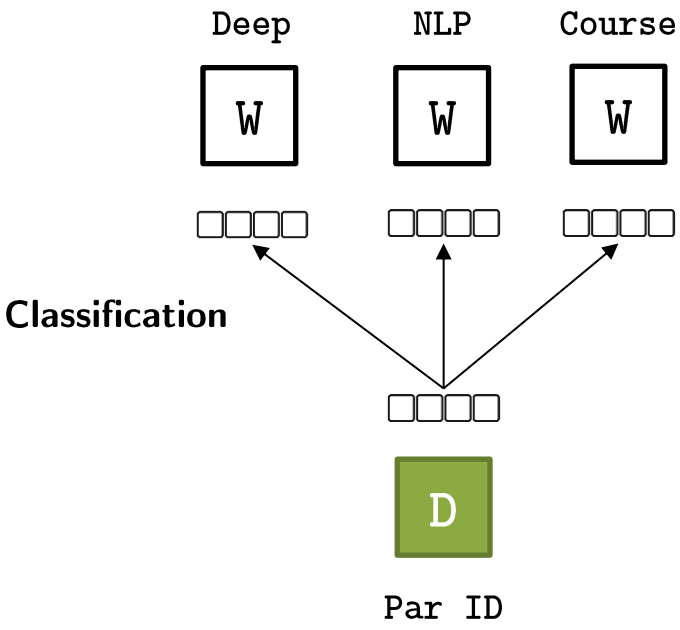
Distributed Bag-of-Words (PV-DBOW)

- Predict a single context word using only the paragraph vector
  - At each iteration of stochastic gradient descent a text window is sampled
  - Then a single random word is sampled from that window forming the aforesaid classification task

Quoc V. Le, Tomás Mikolov. Distributed Representations of Sentences and Documents. ICML 2014: 1188-1196

Doc2Vec - DBOW

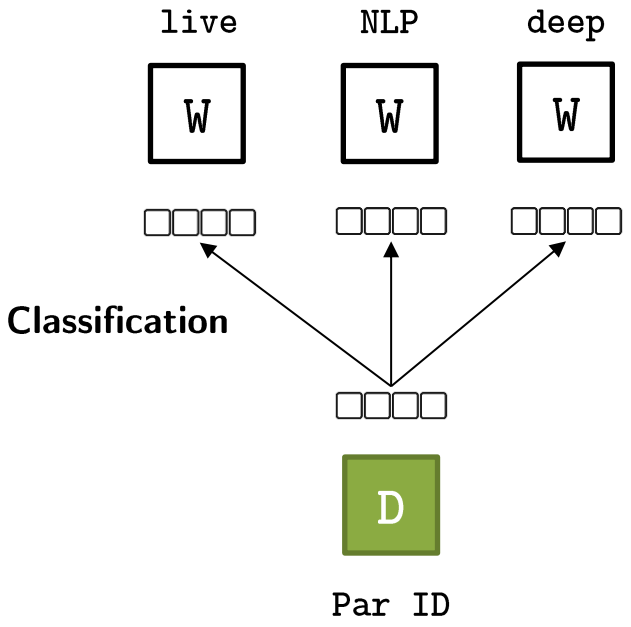
- **DBOW**: it is similar to the Skip-Gram architecture of Word2Vec



Paragraph: Deep NLP course is live

Doc2Vec - DBOW

- **Training Phase:**
  - Represent both paragraphs and words with unique IDs
  - Given paragraph ID:
    - **Input:** one-hot encoded representation of the paragraph ID.
    - **Output:** is one hot encoded representation of a randomly selected word.
  - The neural network transforms one hot encoded representation to paragraph vector. It is passed through a softmax layer. Both set of weights are adjusted during training phase.

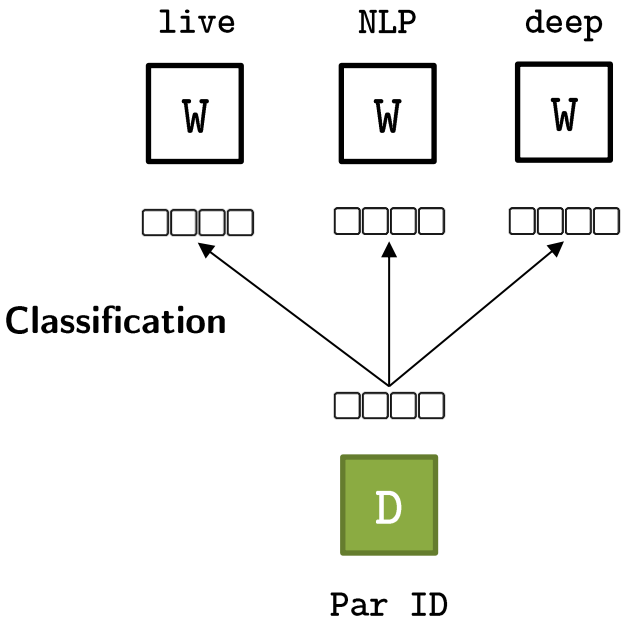


After training: all paragraph IDs are mapped to a new space such that probabilities for **randomly selected words** in each document are maximized starting from that vector space representation to softmax output.

Doc2Vec - DBOW

Inference:

- What vector space representation is most appropriate for an input document
  - Use the same (trained) set of weights from hidden space to output layer.
  - The weights from hidden layer to softmax output are frozen
- Select a word from new document at random
- Start with a random representation for the document vector
- Adjust the randomly initialized weights such that softmax probability is maximized for the selected word.



An additional **hyperparameter** is required to set the number of update steps. It represents the number of updates to the paragraph vector before obtaining the final embedding.

Doc2Vec - implementation

GENSIM
topic modelling for humans

```python
from gensim.models.doc2vec import Doc2Vec, TaggedDocument
from nltk.tokenize import word_tokenize
from sklearn.metrics.pairwise import cosine_similarity

data = ["Deep NLP course is live.",
        "It's a course for PoliTO students.",
        "It's a course for Master students.",
        "PoliTO is located in Turin."]

tagged_data = [TaggedDocument(words=word_tokenize(_d.lower()), tags=[str(i)]) for i, _d in enumerate(data)]


max_epochs = 100
emb_size = 200
alpha = 0.01

model = Doc2Vec(documents=tagged_data,
                vector_size=emb_size,
                alpha=alpha,
                min_alpha=0.0001,
                min_count=1,
                dm =1,
                epochs=max_epochs)

model.save("d2v.model")
print("Model Saved")

vector_1 = model.infer_vector(["nlp", "course"])
vector_2 = model.infer_vector(["polito", "students"])
print (cosine_similarity(vector_1.reshape(1, -1), vector_2.reshape(1, -1)))
```

Represents a document along with a tag, input document format for Doc2Vec

Remove words with the number of occurrences lower than this threshold

Doc2Vec - implementation

GENSIM
topic modelling for humans

```python
from gensim.models.doc2vec import Doc2Vec, TaggedDocument
from nltk.tokenize import word_tokenize
from sklearn.metrics.pairwise import cosine_similarity

data = ["Deep NLP course is live.",
        "It's a course for PoliTO students.",
        "It's a course for Master students.",
        "PoliTO is located in Turin."]

tagged_data = [TaggedDocument(words=word_tokenize(_d.lower()), tags=[str(i)]) for i, _d in enumerate(data)]


max_epochs = 100
emb_size = 200
alpha = 0.01

model = Doc2Vec(documents=tagged_data,
                vector_size=emb_size,
                alpha=alpha,
                min_alpha=0.0001,
                min_count=1,
                dm =1,
                epochs=max_epochs)

model.save("d2v.model")
print("Model Saved")

vector_1 = model.infer_vector(["nlp", "course"])
vector_2 = model.infer_vector(["polito", "students"])
print (cosine_similarity(vector_1.reshape(1, -1), vector_2.reshape(1, -1)))
```

Embedding size

Flag to use DM algorithm

Doc2Vec - implementation

GENSIM
topic modelling for humans

```python
from gensim.models.doc2vec import Doc2Vec, TaggedDocument
from nltk.tokenize import word_tokenize
from sklearn.metrics.pairwise import cosine_similarity

data = ["Deep NLP course is live.",
        "It's a course for PoliTO students.",
        "It's a course for Master students.",
        "PoliTO is located in Turin."]

tagged_data = [TaggedDocument(words=word_tokenize(_d.lower()), tags=[str(i)]) for i, _d in enumerate(data)]


max_epochs = 100
emb_size = 200
alpha = 0.01

model = Doc2Vec(documents=tagged_data,
                vector_size=emb_size,
                alpha=alpha,
                min_alpha=0.0001,
                min_count=1,
                dm =1,
                epochs=max_epochs)

model.save("d2v.model")
print("Model Saved")

vector_1 = model.infer_vector(["nlp", "course"])
vector_2 = model.infer_vector(["polito", "students"])
print (cosine_similarity(vector_1.reshape(1, -1), vector_2.reshape(1, -1)))
```

Initial learning rate

Learning rate will linearly drop to min_alpha as training progresses

Doc2Vec - implementation

GENSIM
topic modelling for humans

```python
from gensim.models.doc2vec import Doc2Vec, TaggedDocument
from nltk.tokenize import word_tokenize
from sklearn.metrics.pairwise import cosine_similarity

data = ["Deep NLP course is live.",
        "It's a course for PoliTO students.",
        "It's a course for Master students.",
        "PoliTO is located in Turin."]

tagged_data = [TaggedDocument(words=word_tokenize(_d.lower()), tags=[str(i)]) for i, _d in enumerate(data)]


max_epochs = 100
emb_size = 200
alpha = 0.01

model = Doc2Vec(documents=tagged_data,
                vector_size=emb_size,
                alpha=alpha,
                min_alpha=0.0001,
                min_count=1,
                dm =1,
                epochs=max_epochs)
```

```python
model.save("d2v.model")
print("Model Saved")
```

Storing the model

Compute cosine similarity between vectors

```python
vector_1 = model.infer_vector(["nlp", "course"])
vector_2 = model.infer_vector(["polito", "students"])
print (cosine_similarity(vector_1.reshape(1, -1), vector_2.reshape(1, -1)))
```

Additional reading on Doc2Vec

- Quoc V. Le, Tomás Mikolov. Distributed Representations of Sentences and Documents. ICML 2014: 1188-1196.
  - Le & Mikolov (2014)

- Download and read the paper
  - https://cs.stanford.edu/~quoc le/paragraph_vector.pdf

- Source code
  - https://github.com/bnosac/doc2vec

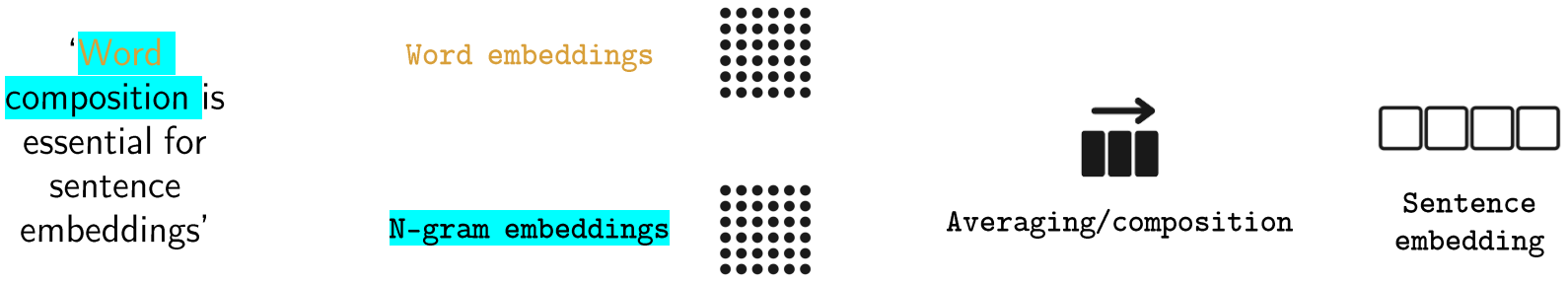
Sent2Vec

- Unsupervised learning of sentence embeddings
  - Distributed representations of sentences
- Based on compositional n-gram features
- It extends FastText and W2V CBOW architectures to sentence-level embeddings
  - the entire sentence is considered as the context window, instead of sampling the fixed-size context window
- It simultaneously trains n-gram embedding vectors and their compositions

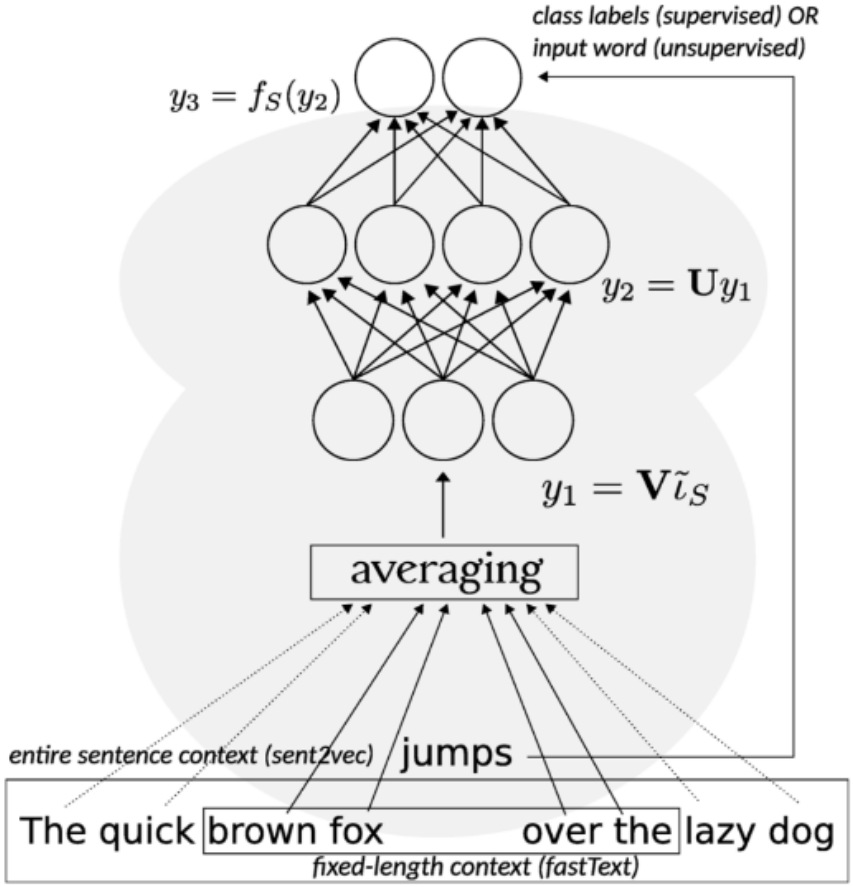
Matteo Pagliardini, Prakhar Gupta, Martin Jaggi. Unsupervised Learning of Sentence Embeddings using Compositional n-Gram Features. NAACL 2018

Sent2Vec

- The sentence embedding is defined as the average of the source word embeddings of its constituent words
- Source embeddings are learned not only for unigrams but also for n-grams present in each sentence
- Final embeddings are obtained by averaging both n-grams and word embeddings



Sent2Vec



Topbot.com (latest access: June 2021)

Sent2Vec

- Simple matrix factor models (bilinear models)

$$\min_{\boldsymbol{U},\boldsymbol{V}} \sum_{S \in \mathcal{C}} f_S(\boldsymbol{U}\boldsymbol{V}\boldsymbol{\iota}_S)$$

Where

- Columns of $U \in R^{k \times h}$ are the target word vectors
- Columns of $V \in R^{h \times |V|}$ denotes the vocabulary
- $\boldsymbol{\iota}_S \in \{0,1\}^{|\mathcal{V}|}$ indicator vector of sentence S

Sent2Vec

- CBOW extension to a sentence context
  - Predict the missing word from the context
- Learn source embeddings for n-grams in the sentence *R(S)*

$$\boldsymbol{v}_S := \frac{1}{|R(S)|} \boldsymbol{V} \boldsymbol{\iota}_{R(S)} = \frac{1}{|R(S)|} \sum_{w \in R(S)} \boldsymbol{v}_w$$

Where

- Columns of $U \in R^{k \times h}$ are the target word vectors
- Columns of $V \in R^{h \times |V|}$ denotes the vocabulary
- $\boldsymbol{\iota}_S \in \{0,1\}^{|\mathcal{V}|}$ indicator vector of sentence *S*

Sent2Vec: optimization task

$$\min_{\boldsymbol{U},\boldsymbol{V}} \sum_{S\in\mathcal{C}} \sum_{w_t\in S} \Big(q_p(w_t)\ell\big(\boldsymbol{u}_{w_t}^\top \boldsymbol{v}_{S\setminus\{w_t\}}\big) + |N_{w_t}| \sum_{w'\in\mathcal{V}} q_n(w')\ell\big(-\boldsymbol{u}_{w'}^\top \boldsymbol{v}_{S\setminus\{w_t\}}\big)\Big)$$

- Binary logistic log function $\ell : x \mapsto \log\left(1 + e^{-x}\right)$
- Negative sampling
  - N_w words sampled negatively for word w_t

Sent2Vec – usage pretrained models

```python
import sent2vec
model = sent2vec.Sent2vecModel()

# pretrained model
model.load_model('model.bin')
emb = model.embed_sentence("NLP is powerful")
embs = model.embed_sentences(["Deep NLP is powerful", "CV vs NLP is the AI battle"])
```

Source: https://github.com/epfml/sent2vec

Sent2Vec – usage pretrained models

Source: https://github.com/epfml/sent2vec

```
# Input file should contain one sentence per line
# Input data should prior pre-processed

./fasttext sent2vec -input wiki_sentences.txt -output my_model -minCount 8 -dim 700 -epoch 9 -lr 0.2 -wordNgrams
2 -loss ns -neg 10 -thread 20 -t 0.000005 -dropoutK 4 -minCountLabel 20 -bucket 4000000 -maxVocabSize 750000
-numCheckPoints 10
```

Path to the output model

Loss function (default: negative sampling)

Number of ngrams dropped when training a sent2vec model

Additional reading on Sent2Vec

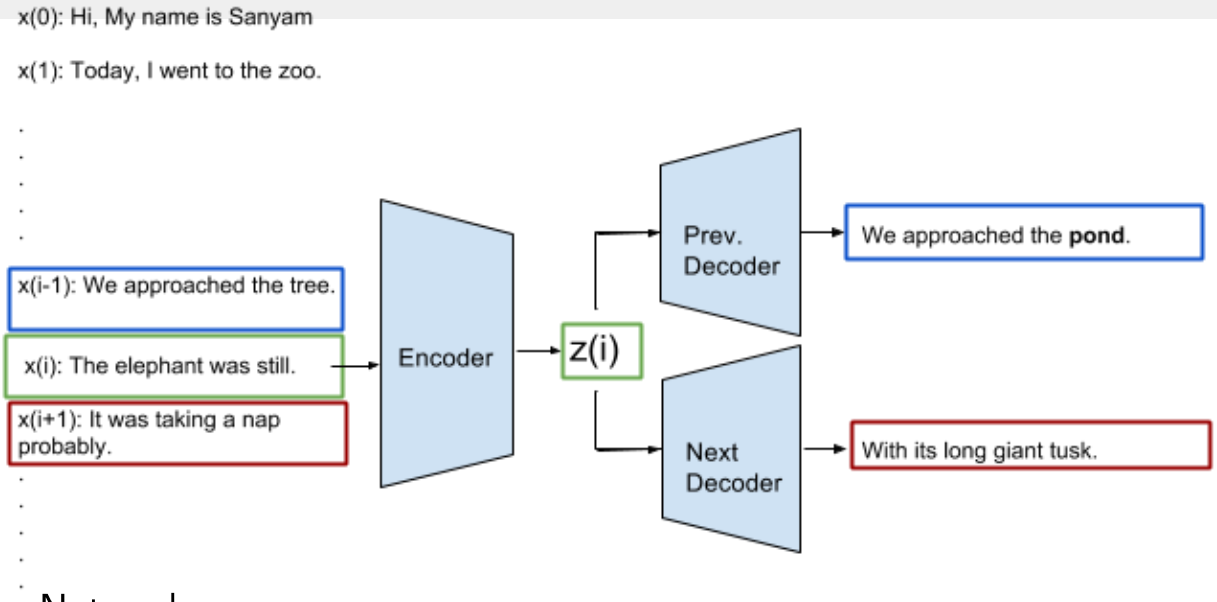
- Matteo Pagliardini, Prakhar Gupta, Martin Jaggi. Unsupervised Learning of Sentence Embeddings using Compositional n-Gram Features. NAACL 2018
  - Proposed by Pagliardini et al. 2018
- Download and read the paper: https://arxiv.org/pdf/1703.02507v3.pdf
- Source code and pretrained models
  - https://github.com/epfml/sent2vec

Skip-thought vectors

- Unsupervised learning of a generic, distributed sentence encoder
- Encoder-decoder model
- Tries to reconstruct the surrounding sentences of an encoded passage
  - Sentences that share semantic and syntactic properties are thus mapped to similar vector representations

Ryan Kiros, Yukun Zhu, Ruslan R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Skip-thought vectors. In Advances in neural information processing systems (NIPS), pages 3294–3302.

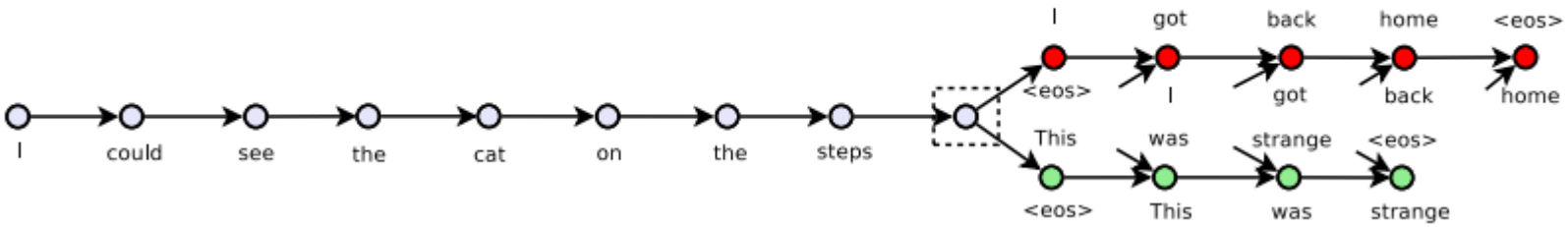
Skip-thought vectors



- Encoder Network
  - Takes the sentence x(i) at index i and generates a fixed-length representation z(i)
  - This is a recurrent network (generally GRU or LSTM) that takes the words in a sentence sequentially
- Previous Decoder Network
  - Takes the embedding z(i) and "tries" to generate the sentence x(i-1)
  - This also is a recurrent network (generally GRU or LSTM) that generates the sentence sequentially
- Next Decoder Network:
  - Takes the embedding z(i) and "tries" to generate the sentence x(i+1)
  - Again a recurrent network similar to the Previous Decoder Network

My thoughts on Skip-Thoughts. Sanyam Agarwal. Research Assistant at IISc, Bangalore. Working in Semi-supervised Learning.2017.

Skip-thought vectors



- Given a tuple (s_{i-1}, s_i, s_{i+1}) of contiguous sentences, with s_{i-1} the i-th sentence of a book
  - s_{i-1}: *I got back home.*
  - s_i : *I could see the cat on the steps.*
  - s_{i+1} : *This was strange.*
  - <eos> : end of sentence token
- s_i is encoded and tries to reconstruct the previous sentence s_{i-1} and next sentence s_{i+1}
- Unattached arrows are connected to the encoder output
- Colors indicate which components share parameters

Ryan Kiros, Yukun Zhu, Ruslan R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Skip-thought vectors. In Advances in neural information processing systems (NIPS), pages 3294–3302.

Additional reading on Skip-thought

- Ryan Kiros, Yukun Zhu, Ruslan R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Skip-thought vectors. In Advances in neural information processing systems (NIPS), pages 3294–3302.
- Download and read the paper: https://arxiv.org/pdf/1506.06726.pdf

InferSent

- Proposed by Facebook researchers
- Source code
  - https://github.com/facebookresearch/InferSent
- Goal: learn universal representations of sentences using using the supervised data of the Stanford Natural Language Inference (SNLI) datasets
  - 570k human-generated English sentence pairs
  - Manually labeled as *entailment, contradiction, or neutral*

Alexis Conneau, Douwe Kiela, Holger Schwenk, Loic Barrault, Antoine Bordes.
Supervised Learning of Universal Sentence Representations from Natural Language Inference Data. EMNLP 2017

InferSent

- Natural Language Inferencing Task
  - Determine whether a “hypothesis” is true, false, or neutral, given a “premise”.
  - It involves high-level reasoning about semantic relationships within sentences.

```
premise: A deep learning model is used to analyze natural language.
hypothesis: NLP tools include deep learning models.
label: Entailment
```

InferSent

- Natural Language Inferencing Task
  - Determine whether a “hypothesis” is true, false, or neutral, given a “premise”.
  - It involves high-level reasoning about semantic relationships within sentences.

```
premise: Syntactic tools are the only way to analyze text
hypothesis: NLP tools include deep learning models.
label: Contradiction
```

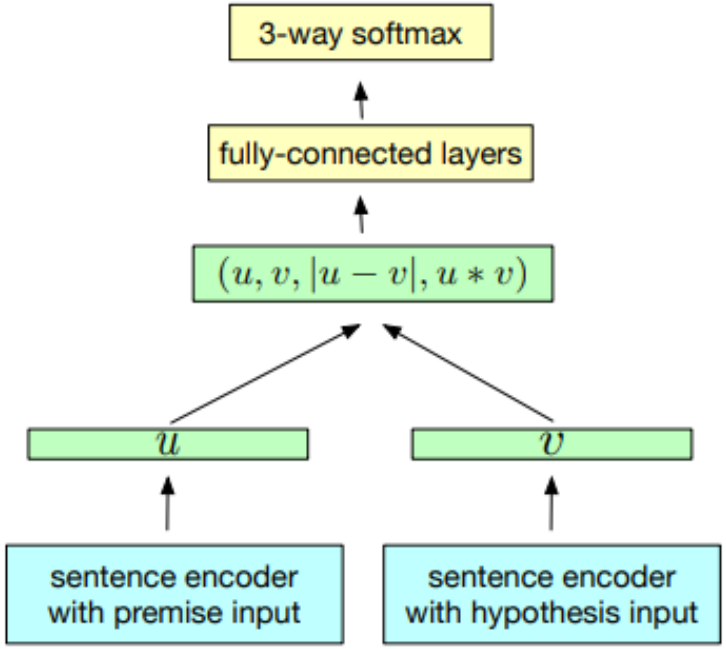
InferSent

- Natural Language Inferencing Task
  - Determine whether a “hypothesis” is true, false, or neutral, given a “premise”.
  - It involves high-level reasoning about semantic relationships within sentences.

```
premise: In CV you can use deep learning to boost performance
hypothesis: NLP tools include deep learning models.
label: Neutral
```

InferSent

- Transfer learning of a supervised model
- The model designed for Textual Entailment Recognition (i.e., infer a directional relation between text fragments) is transferred to learn universal sentence embeddings



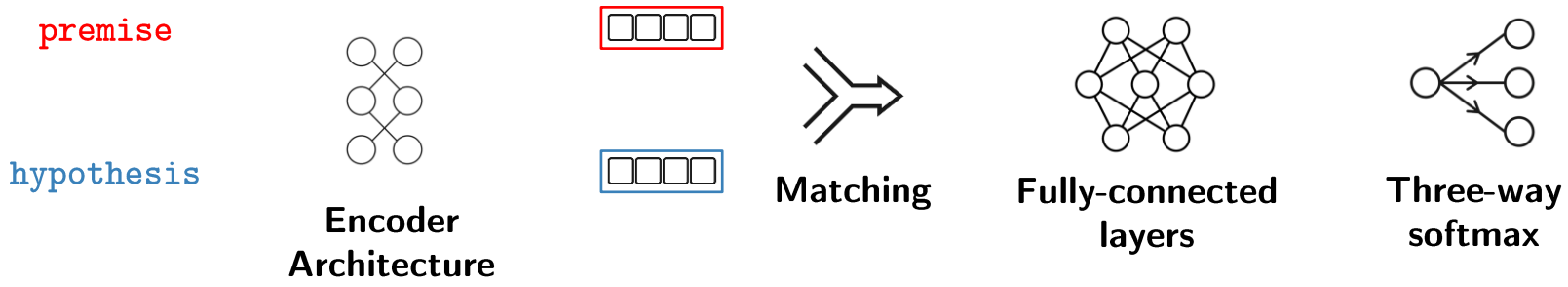
Text entailment examples

- Positive TE (text entails hypothesis)
  - text: *If you help the needy, God will reward you.*
  - hypothesis: *Giving money to a poor man has good consequences.*
- Negative TE (text contradicts hypothesis)
  - text: *If you help the needy, God will reward you.*
  - hypothesis: *Giving money to a poor man has no consequences.*
- Non-TE (text does not entail nor contradict)
  - text: *If you help the needy, God will reward you.*
  - hypothesis: *Giving money to a poor man will make you a better person.*

InferSent

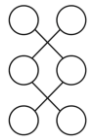
- **Training:**
  - **Input:** premise and hypothesis
  - **Output:** entailment, contradiction, or neutral

- **Embedding matching:**
  - Concatenation
  - Element-wise product
  - Absolute element-wise difference



InferSent

- **Embedding model:**
  1. LSTM
  2. GRU
  3. Bidirectional GRU + concat
  4. Bidirectional LSTM with average pooling
  5. Bidirectional LSTM with max pooling
  6. Self-attentive network
  7. Hierarchical CNN



Embedding model

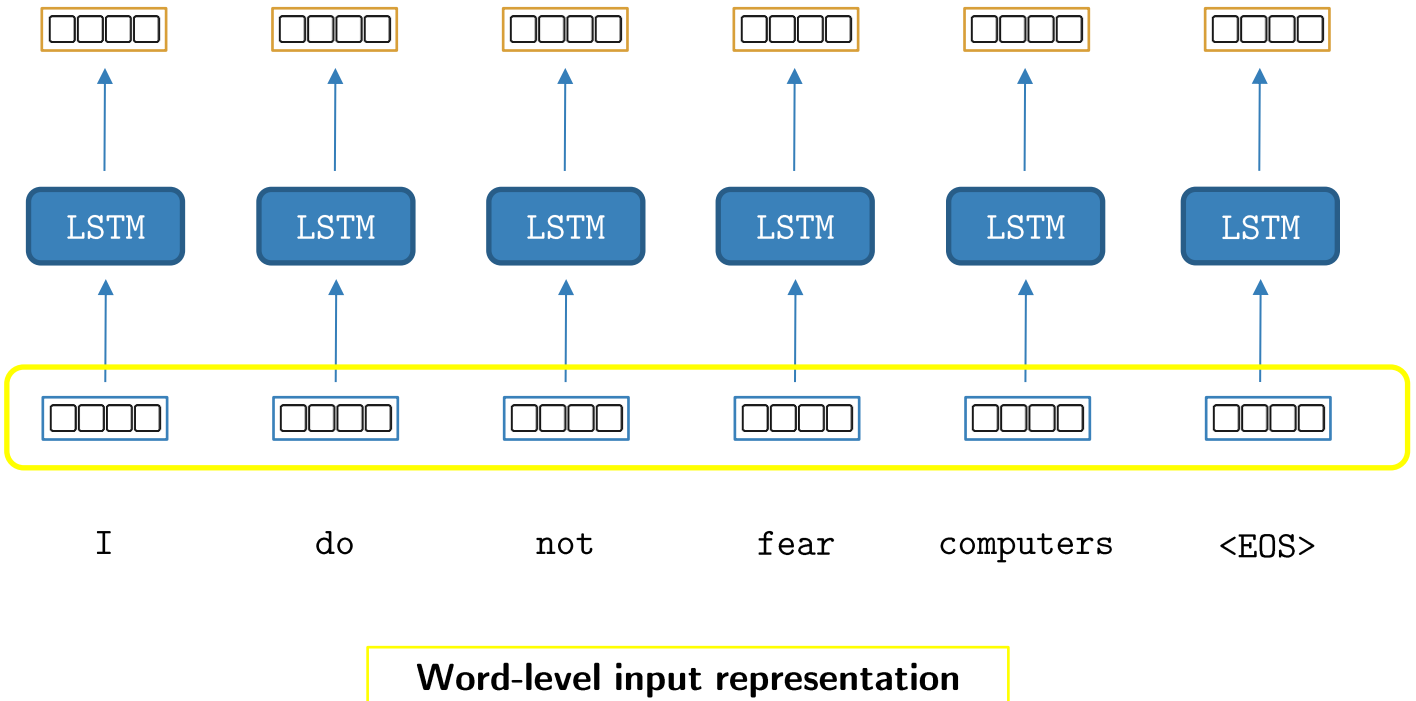
InferSent - preliminaries

```
# preliminaries

git clone https://github.com/facebookresearch/InferSent.git
cd InferSent

mkdir GloVe
curl -Lo GloVe/glove.840B.300d.zip http://nlp.stanford.edu/data/glove.840B.300d.zip
unzip GloVe/glove.840B.300d.zip -d GloVe/
mkdir fastText
curl -Lo fastText/crawl-300d-2M.vec.zip https://dl.fbaipublicfiles.com/fasttext/vectors-english/crawl-
300d-2M.vec.zip
unzip fastText/crawl-300d-2M.vec.zip -d fastText/

# download model
curl -Lo encoder/infersent1.pkl https://dl.fbaipublicfiles.com/infersent/infersent1.pkl
```

https://github.com/facebookresearch/InferSent

Why word embedding model?

InferSent - preliminaries



InferSent - usage

```
from models import InferSent

VERSION = 2
MODEL_PATH = 'encoder/infersent%s.pkl' % VERSION
params_model = {'bsize': 64, 'word_emb_dim': 300, 'enc_lstm_dim': 2048,
                'pool_type': 'max', 'dpout_model': 0.0, 'version': VERSION}
infersent = InferSent(params_model)
infersent.load_state_dict(torch.load(MODEL_PATH))

W2V_PATH = 'fastText/crawl-300d-2M.vec'
infersent.set_w2v_path(W2V_PATH)

# sente
infersent.build_vocab(sentences, tokenize=True)
embeddings = infersent.encode(sentences, tokenize=True)
```

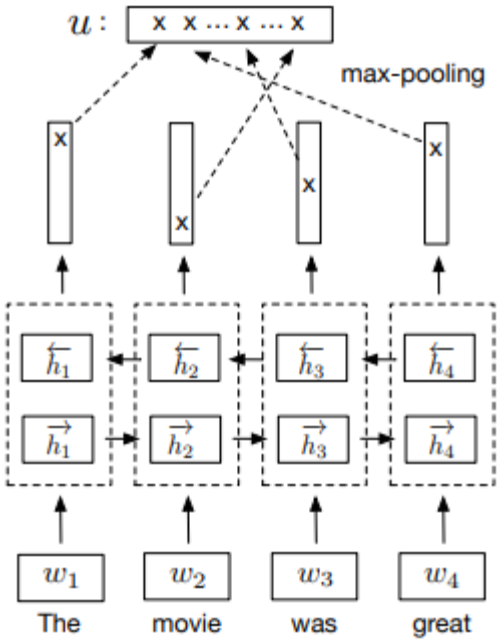
https://github.com/facebookresearch/InferSent

InferSent

- Baseline architecture: LSTM
  - For a sequence of T words (w_1, ... , w_T) the network computes a set of T hidden representations h_1, . . . , h_T
    - $h_t = \overrightarrow{\text{LSTM}}(w_1, \ldots, w_T)$ (or using GRU units instead)
  - A sentence is represented by the last hidden vector h_T
  - Max pooling
    - select the maximum value over each dimension of the hidden units

InferSent

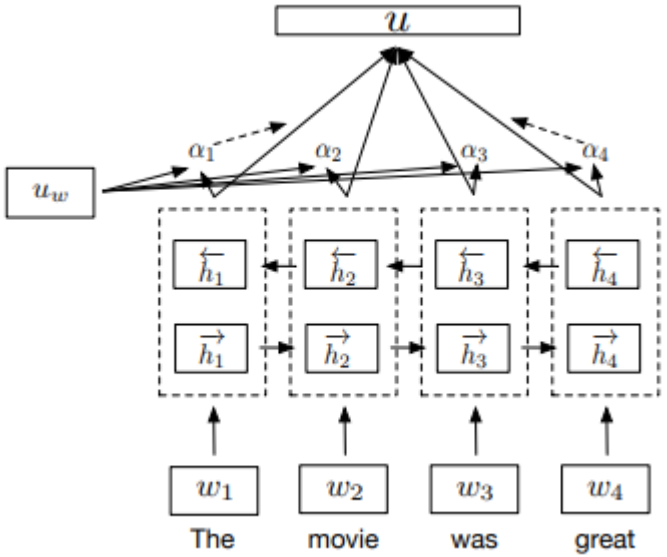
- Bidirectional LSTM
  - The BiLSTM computes a set of T vectors $\{h_t\}_t$
  - For $t \in [1, \ldots, T]$, h_t is the concatenation of a forward LSTM and a backward LSTM that read the sentences in two opposite directions



$$\overrightarrow{h_t} = \overrightarrow{\text{LSTM}}_t(w_1, \ldots, w_T)$$
$$\overleftarrow{h_t} = \overleftarrow{\text{LSTM}}_t(w_1, \ldots, w_T)$$
$$h_t = [\overrightarrow{h_t}, \overleftarrow{h_t}]$$

InferSent

- Self-attentive network
  - It uses an attention mechanism over the hidden states of a BiLSTM to generate a representation u of an input sentence
    - $\{h_1, \ldots, h_T\}$ are the output hidden vectors of a BiLSTM
    - tanh() is an affine transformation that outputs a set of keys
    - The $\{\alpha_i\}$ represents the score of similarity between the keys and a learned context query vector u_w
  - The final sentence representation u is a weighted linear combination of the hidden vectors



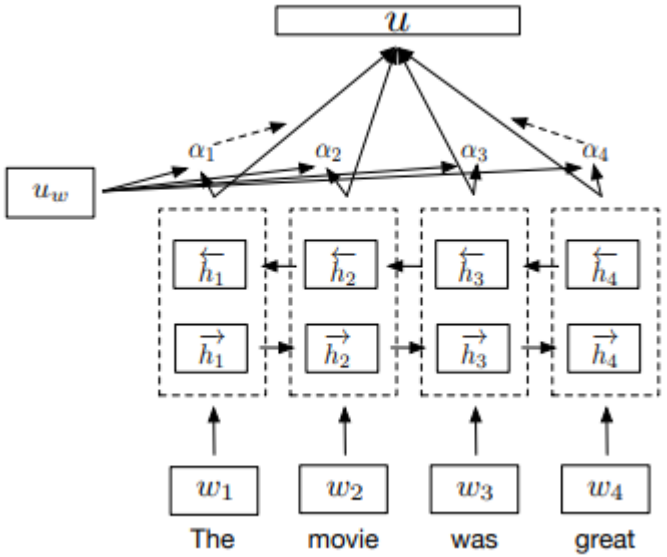
$$\bar{h}_i = \tanh(W h_i + b_w)$$

$$\alpha_i = \frac{e^{\bar{h}_i^T u_w}}{\sum_i e^{\bar{h}_i^T u_w}}$$

$$u = \sum_t \alpha_i h_i$$

InferSent

- Self-attentive network
  - From the multiple views of the input sentence the model can learn which part of
  - the sentence is important for the given task
  - 4 context vectors u_w which generate 4 representations that are then concatenated to obtain the sentence representation u



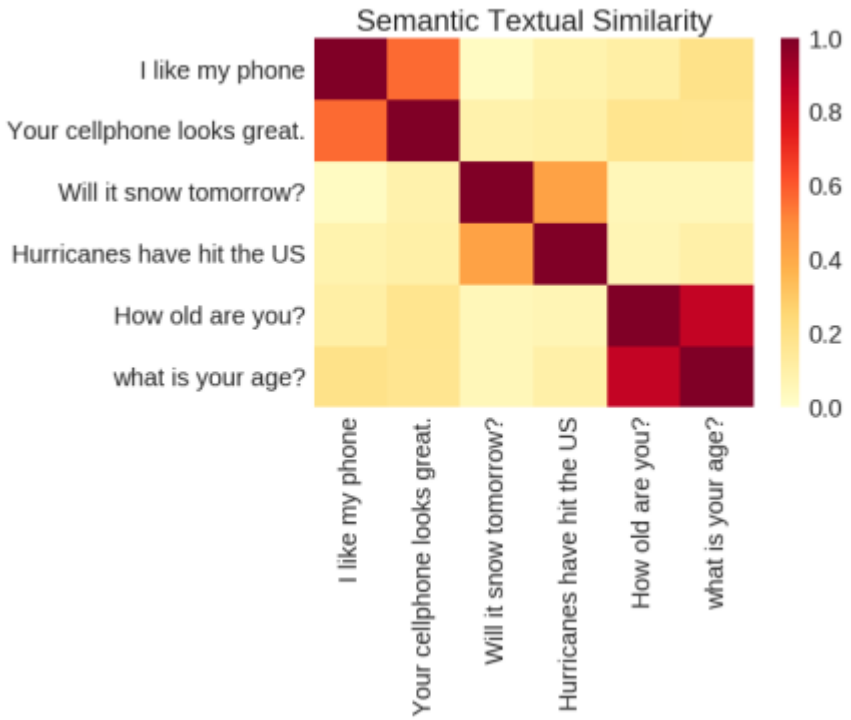
$$\bar{h}_i = \tanh(W h_i + b_w)$$

$$\alpha_i = \frac{e^{\bar{h}_i^T u_w}}{\sum_i e^{\bar{h}_i^T u_w}}$$

$$u = \sum_t \alpha_i h_i$$

Universal Sentence Encoder

- Proposed by Google Inc.
- Supervised models for encoding sentences into embedding vectors that specifically target transfer learning to other NLP tasks
- Task
  - Semantic text similarity
- Project
  - https://tfhub.dev/google/universal-sentence-encoder/1



Universal Sentence Encoder. Daniel Cera, Yinfei Yanga, Sheng-yi Konga, Nan Huaa, Nicole Limtiacob, Rhomni St. Johna, Noah Constanta, Mario Guajardo-Cespedesa, Steve Yuanc, Chris Tara, Yun-Hsuan Sunga, Brian Stropea, Ray Kurzweil.

Lecture goals

- From word to sentences
- Sentence-level embedding models
  - Doc2Vec
  - Sent2Vec
  - InferSent
  - Universal Sentence Encoder
- **Embedding model evaluation**
  - Intrinsic evaluation
  - Extrinsic evaluation
- Bias in word embedding models

Evaluation of text encoders

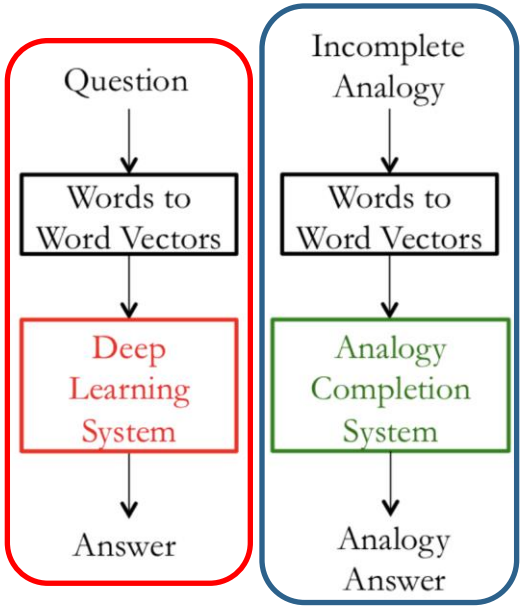
- Intrinsic evaluation
  - Generic evaluation of the quality and coherence of the vector space
  - It is independent from model performance on downstream tasks

- Extrinsic Evaluation
  - Assess model perfomance when used as preliminary step in a downstream task (e.g., machine translation)

Intrinsic Evaluation

- Evaluation on a specific, intermediate task
  - Fast to compute performance
- Help to understand vector subsystem
  - Needs positive correlation with real task to determine usefulness

Extrinsic Evaluation
Expensive training of the end-to-end system



Intrinsic Evaluation
a simple intrinsic evaluation technique which can provide a measure of "goodness" of the word to word vector subsystem.

https://cs224d.stanford.edu/lecture_notes/notes2.pdf

Intrinsic Evaluation - *Word Vector Analogies*

In a word vector analogy, we are given an incomplete analogy of the form:

$$a : b = c : ?(d)$$

The goal is to obtain the following relation:

$$b - a = d - c$$

This implies:

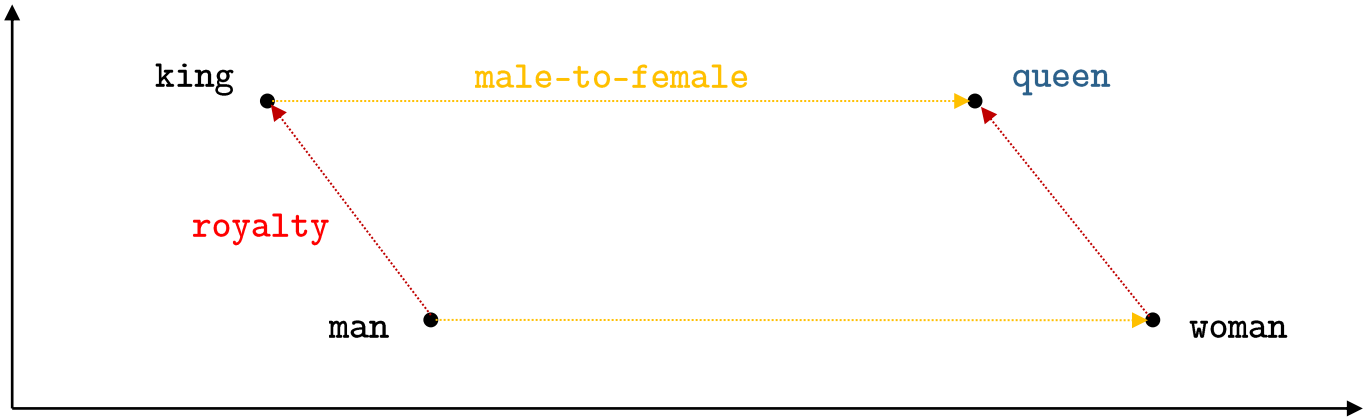
$$b - a + c = d$$

Identify the vector **d** which maximizes the normalized dot-product between the two word vectors (i.e. cosine similarity).

Intrinsic Evaluation - *Word Vector Analogies*

In a word vector analogy, we are given an incomplete analogy of the form:

$$\texttt{a : b = c : ?}$$

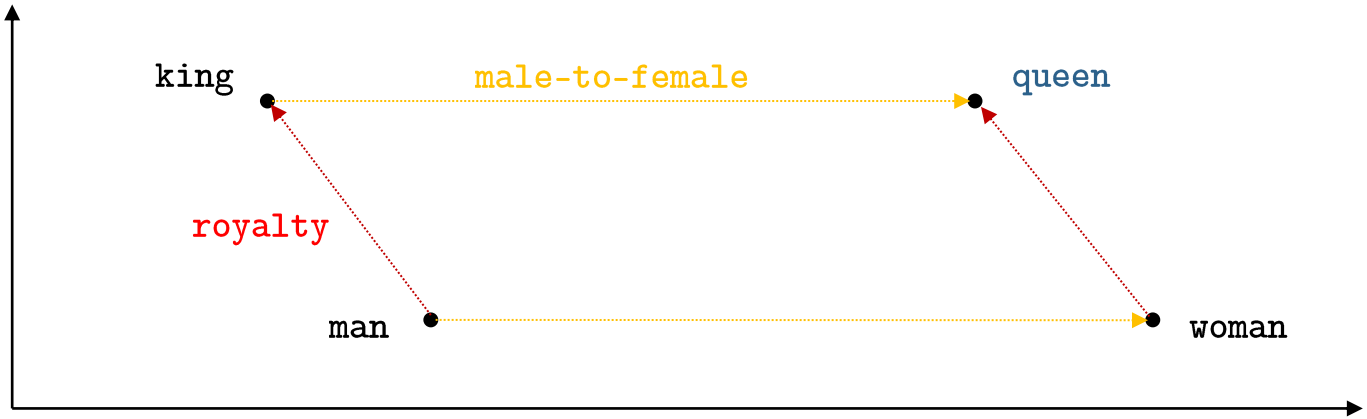


Intrinsic Evaluation - *Word Vector Analogies*

In a word vector analogy, we are given an incomplete analogy of the form:

$$\texttt{man : king = woman : queen}$$

$$\texttt{king - man + woman = queen}$$



Intrinsic Evaluation - *Word Vector Analogies*

An example of analogy dataset could be:

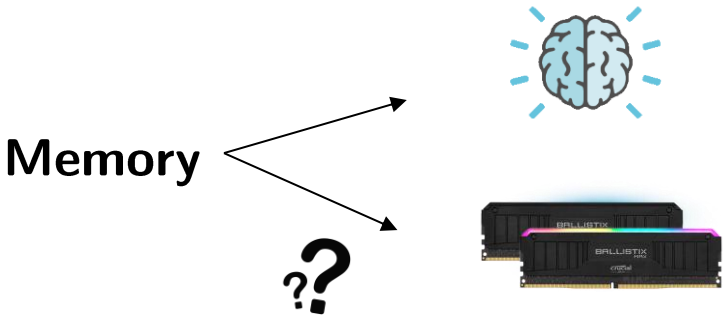
Input	Result
`Italy : Rome = France : (?)`	`Paris`
`Italy : Rome = Belgium : (?)`	`Bruxelles`
`Italy : Rome = Germany : (?)`	`Berlin`
`Italy : Rome = Australia : (?)`	`Canberra`
`Italy : Rome = Greece : (?)`	`Athens`
`...`	`...`

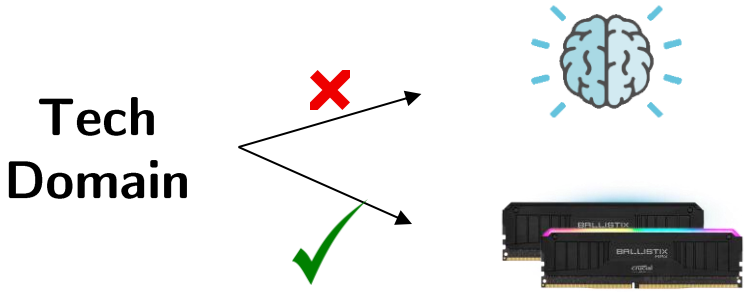
What type of similarity?

- **Property-based:** if they share many properties (e.g., bike-motorcycle)
- **Meronymy:** a meronym denote a part and a holonym denoting a whole (e.g., wheel-bike)
- **Antonymy:** the semantic relationship between words that have opposite meanings (e.g., good-bad)
- **Topic relation:** both words refer to the same topic (e.g., zebra-zoo)

Limitations

- It only considers the **attributional** similarity between words. Irrelevant for some NLP tasks (e.g., North, South in QA systems)
- **Hubness**: an **hub** in the semantic space is a word that has high semantic similarity with a large number of words.
- **Polysemous nature** of words: a single word can have multiple meanings in different domains

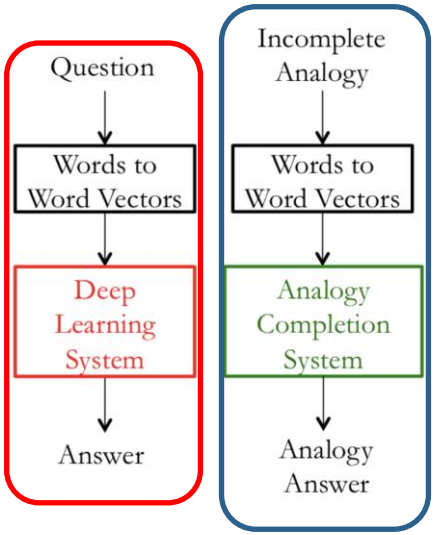




Extrinsic Evaluation

- Evaluation on a real task
- Can be slow to compute performance
- Unclear if subsystem is the problem
- If replacing subsystem improves performance, the change is likely good

Extrinsic evaluation
Evaluation of a set of word vectors generated by an embedding technique on the real task at hand.

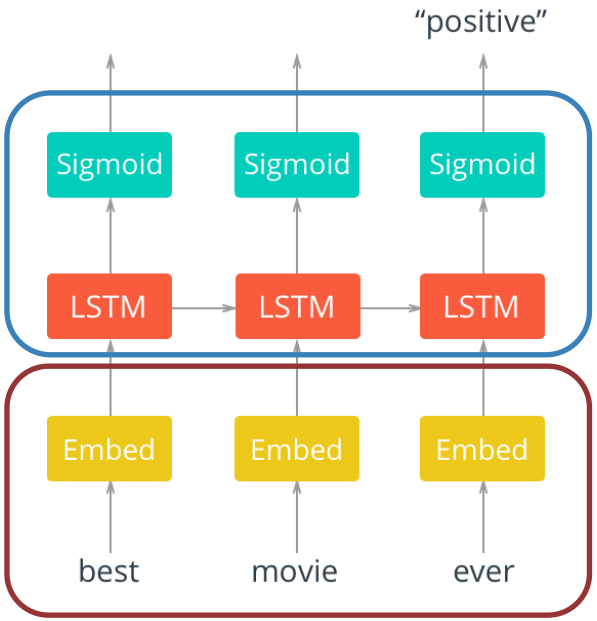


Intrinsic Evaluation
Low correlation with final system performance

https://cs224d.stanford.edu/lecture_notes/notes2.pdf

Extrinsic Evaluation

- Most NLP extrinsic tasks can be formulated as classification tasks
- **Example:** given a sentence, we can classify the sentence to have positive, negative or neutral sentiment

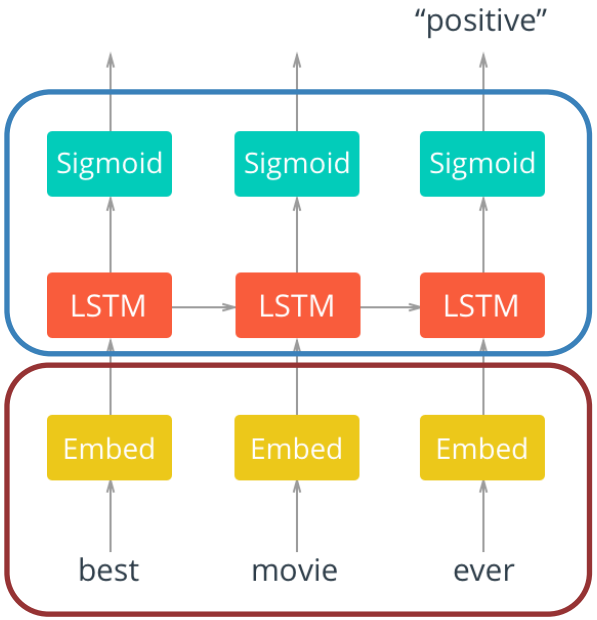


Remaining part of the final architecture

1. Word2Vec
2. Glove
3. FastText
4. ...

Extrinsic Evaluation

- It could be **unclear** whether performance improvement is related to word embedding models or other model settings
- Usually, the word vectors used in extrinsic tasks are initialized by optimizing them over a **simpler intrinsic task**



Remaining part of the final architecture

1. Word2Vec
2. Glove
3. FastText
4. ...

Lecture goals

- From word to sentences
- Sentence-level embedding models
  - Doc2Vec
  - Sent2Vec
  - InferSent
  - Universal Sentence Encoder
- Embedding model evaluation
  - Intrinsic evaluation
  - Extrinsic evaluation
- **Bias in word embedding models**

Bias in embedding models

- Word embeddings model text meaning, however, text could incorporate implicit biases and stereotypes

```
man : king = woman : queen
man : programmer = woman : ?
```

- **Allocation harm:** when a system allocate resources (jobs or credit) unfairly to different groups.

Example of bias in embedding models

- In [1] the authors used GloVe embeddings to analyze racial-related bias in the embedding model
- They found that African-American names (Leroy, Shaniqua) had higher cosine similarity with unpleasant words if compared with European-American names (Brad, Greg)
- **Representational harm:** when a system demeans or ignores some social groups.
- **Bias** mitigation or reduction is an open (and discussed) problem in NLP research

[1] Caliskan, A., Bryson, J. J., & Narayanan, A. (2017). Semantics derived automatically from language corpora contain human-like biases. *Science*, *356* (6334), 183-186.

Acknowlegdements and copyright license

- Copyright licence
  - Attribution + Noncommercial + NoDerivatives

- Acknowledgements
  - I would like to thank Dr. Lorenzo Vaiani, who collaborated to the writing and revision of the teaching content
- Affiliation
  - The author and his staff are currently members of the Database and Data Mining Group at Dipartimento di Automatica e Informatica (Politecnico di Torino) and of the SmartData interdepartmental centre
    - https://dbdmg.polito.it
    - https://smartdata.polito.it

Thank you!